

Allgemeines

Im Rahmen des Kurses **Softwarequalität in der komponentenbasierten Entwicklung** sind 20% der Note über die Teilnahme an einem Seminar zu erbringen. Dieses Seminar teilt sich dabei in zwei Teile auf. Der erste Teil des Seminars fokussiert sich auf die Entwicklung von Software mithilfe von industrie-üblichen Frameworks zur komponentenbasierten Entwicklung. Im zweiten Teil des Seminars liegt der Fokus, dann auf der Anwendung verschiedener Softwarequalitätsmaßnahmen. Genauere Informationen zur Gesamtbewertung dieses Kurses und den weiteren Bestandteilen der Note können Sie den Vorlesungsunterlagen entnehmen. Sollten sich Fragen zum Seminar ergeben, haben Sie jederzeit die Möglichkeit Fragen in den Seminar-Veranstaltungen und per E-Mail zu stellen.

Ausgangssituation

Ein Kunde hat Sie mit der Erstellung eines *Proof of Concept* bzw. eines Prototyps beauftragt. Der Kunde besitzt zwei speicherprogrammierbare Steuerungen (SPS), wobei eine SPS kontinuierlich Temperatursensorwerte ausliest und die andere SPS eine Lichtsteuerung bedient. Zu den beiden SPS-Typen werden die folgenden Daten versendet:

Wago 750:

status (Integer) , als 2 Byte BinaryArray gesehen, ergibt dieser Wert die Leuchten die aktuell an/aus sind.

Siemens S7-1500:

Ist-Temperatur (Double)

Soll-Temperatur (Double)

Differenz-Temperatur (Double)

Ihr *Proof of Concept* bzw. Prototyp soll dem Kunden zeigen, dass es mit aktuell gängigen Schnittstellentechnologien, wie MQTT und REST und der Nutzung mehrerer Microservices möglich ist, eine Softwarekomposition zu erstellen, mit der die zuletzt gemessenen und gesendeten Daten während des laufenden Betriebs der SPS in einem Frontend darstellbar gemacht und die SPS auch angesteuert werden können. Für die Datenübertragung von der SPS hat sich der Kunde schon für eine Lösung mithilfe des MQTT-Brokers *Mosquitto* festgelegt. Der Broker ist nur über eine Authentifizierung mit Benutzername und Passwort über das Internet unter der folgenden Adresse/Port erreichbar: **sr-labor.ddns.net:1883**. Für diesen MQTT-Broker ist ihr Benutzername ihre Hochschul-Mailadresse und das Passwort ist ihre Matrikelnummer. Der Broker bietet die folgenden Topics für die beiden SPS-Typen an:

Wago 750:

Wago750/Status

Wago750/Control

Siemens S7-1500:

S7_1500/Temperatur/Ist

S7_1500/Temperatur/Soll

S7_1500/Temperatur/Differenz

Weiterhin bietet der Broker untem dem folgenden Topic **Positionsdaten** eines GPS-Empfängers im JSON-Format an:

LocCheck/862315069391414/Data/GNSS

Sollte die SPS mal nicht erreichbar sein, so erhalten Sie Werte zum Testen über das Topic Random/Integer!

Vorbereitung

Erstellen Sie zu der gegebenen Aufgabenstellung ein grobes Architekturdigramm, in dem Sie alle Komponenten, sowie die verwendeten Technologien für die Kommunikation und Kommunikationswege einzeichnen. Im ersten Seminartermin wird dieses Architekturdigramm in kurzen Einzelgesprächen mit Ihnen besprochen.

Die Daten, welche von den speicherprogrammierbaren Steuerungen (SPS) versendet werden, sollen in einer Datenbank persistiert werden. Als Datenbank sollen Sie an dieser Stelle eine dokumentenbasierte Datenbank, wie *MongoDB*¹ verwenden. Für dieses Seminar ist es ausreichend, wenn die Datenbank lokal auf ihrem Rechner läuft. Sie können dazu eine gängige Container-Virtualisierung, wie bspw. Docker² verwenden. Es kann an dieser Stelle nützlich sein, sich einen MongoDB-Client wie bspw. *MongoDB Compass*³ zu installieren, um schnell grafischen Zugang zur Datenbank zu haben, um bspw. zu prüfen, ob Daten gespeichert werden. Bedenken Sie, dass Sie gegebenenfalls einen Benutzer für die Datenbank anlegen müssen, in der die Daten der speicherprogrammierbaren Steuerungen gespeichert werden.

Aufgabe 1 (1 Punkt)

Installieren Sie einen beliebigen MQTT-Client ihrer Wahl, wie bspw. MQTXX⁴ und testen Sie ihre Zugangsdaten zum MQTT-Broker. Den Empfang von Nachrichten können Sie bspw. mit dem Topic *Wago750/Status* ausprobieren. Implementieren Sie einen Microservice, der sich mit dem zuvor genannten MQTT-Broker verbinden kann und die dort ankommenden Nachrichten annehmen kann. Dazu sollen Sie unter Java das Spring Boot-Framework verwenden. Spring selbst, bietet über eine Bibliothek MQTT Support⁵ an, um Nachrichten von einem MQTT-Broker zu empfangen oder über diesen zu versenden.

Aufgabe 2 (2 Punkte)

Verwenden Sie die, aus dem Kapitel **Vorbereitung**, angelegte Datenbank nun, um die Daten, die von ihrem MQTT-Consumer-Microservice aus **Aufgabe 1** konsumiert

¹MongoDB: <https://www.mongodb.com/>

²Docker: <https://www.docker.com/>

³MongoDB-Compass: <https://www.mongodb.com/products/tools/compass>

⁴MQTXX: <https://mqtxx.app/>

⁵MQTTSupport: <https://docs.spring.io/spring-integration/reference/mqtt.html>

werden, abzuspeichern. Erweitern Sie den MQTT-Consumer-Microservice also entsprechend. Auch hier bietet das Spring-Framework eine entsprechende Integration⁶ an, um die Nutzung in Kombination mit einer *MongoDB* wesentlich zu vereinfachen. Die Daten der beiden speicherprogrammierbaren Steuerungen sowie die Positionsdaten sollen im JSON-Format in unterschiedlichen Collections in der Datenbank gespeichert werden. Mit einem MongoDB-Client wie *MongoDB-Compass* können Sie beim Ausprobieren ihrer Umsetzung sehr leicht testen, ob Daten in die Datenbank geschrieben werden.

Aufgabe 3 (2,5 Punkte)

Implementieren Sie nun einen neuen Microservice, der als Restful Web Service, die gespeicherten Daten aus der MongoDB abfragen kann und über die nachfolgend festgelegten GET-Routen zur Verfügung stellen kann.

Wago 750:

- GET-Route, die den aktuellsten Datenpunkt als JSON zurückgibt.
- (falls benötigt) GET-Route, die den aktuellsten Datenpunkt als JSON zurückgibt, dabei aber den Licht-Status als BinaryArray beinhaltet.

Siemens S7-1500:

- GET-Route/n, die für jeden Typ des Temperaturwerts, also Ist-Wert, Soll-Wert, Differenz-Wert, den aktuellsten Datenpunkt als JSON zurückgibt.
- GET-Route/n, die für jeden Typ des Temperaturwerts, also Ist-Wert, Soll-Wert, Differenz-Wert, alle in der Datenbank enthaltenen Datenpunkte, als JSON zurückgibt.

Installieren Sie sich einen beliebigen REST-Client ihrer Wahl, wie bspw. *Insomnia*⁷, und hinterlegen Sie dort die implementierten Routen. So können Sie die korrekte Funktion der von Ihnen implementierten Routen testen.

Aufgabe 4 (2,5 Punkte)

Implementieren Sie mit einem Frontend-Framework (und Programmiersprache) ihrer Wahl eine grafische Nutzeroberfläche um die Daten der SPS in einem Frontend anzuzeigen. Dazu sollte das von Ihnen ausgewählte Frontend-Framework und dann auch ihre Implementierung in der Lage sein, den von Ihnen entwickelten Restful Web Service aus **Aufgabe 4** aufzurufen, um die entsprechenden Daten zu erhalten. Das Design des Frontends darf dabei recht grob ausfallen. Ihr implementiertes Frontend sollte allerdings die folgenden Eigenschaften erfüllen:

- Es gibt ein grafisches User Interface für die **Wago 750**, ein grafisches User Interface für die **Siemens S7-1500** sowie ein grafisches User Interface für die **Positionsdaten**.

⁶Spring MongoDB Integration: <https://spring.io/guides/gs/accessing-data-mongodb>

⁷Insomnia REST-Client: <https://insomnia.rest/>

- Im grafischen User Interface für die **Wago 750** wird der aktuelle Status der Lichtsteuerung in grafischen Elementen angezeigt.
- Im grafischen User Interface für die **Siemens S7-1500** wird jeweils der letzte bekannte Wert des Ist-Werts, des Soll-Werts und des Differenz-Werts angezeigt.
- Im grafischen User Interface für die **Positionsdaten** werden die Positionskoordinaten (Longitude und Latitude) sowie die Höhe (Altitude) angezeigt oder alternativ auf einer Karte dargestellt.

Ein Beispiel für die beiden grafischen User Interfaces für beide speicherprogrammierbaren Steuerungen finden Sie auf der letzten Seite der Aufgabenstellung.

Aufgabe 5 (2 Punkte)

Implementieren Sie abschließend den folgenden Anwendungsfall:

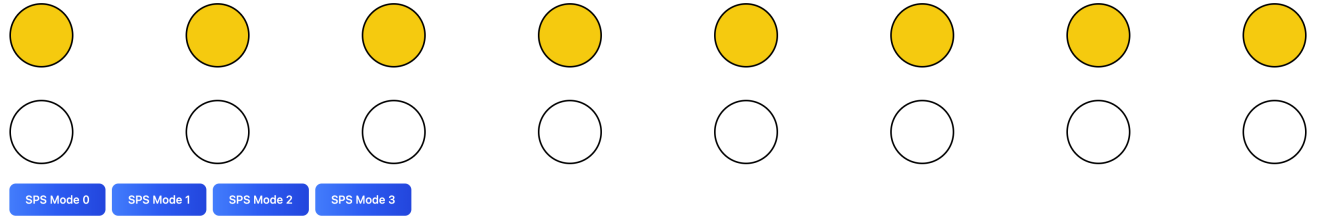
Die **Wago 750** SPS kann mit einem Kommando gesteuert werden. An die SPS können über MQTT an das Topic: *Wago750/Control* die Integer-Werte 0, 1, 2 oder 3 gesendet werden. Wenn die SPS einen dieser Control-Befehle erhält, wechselt sie ihren Modus. Beim Control-Befehl 0 geht die SPS in einen Ruhemodus und schickt keine weiteren Daten mehr. Bei den Control-Befehlen 1, 2 oder 3, wechselt die SPS den Beleuchtungsmodus.

Fügen Sie in ihrem Frontend also die entsprechenden GUI-Elemente hinzu, um das Senden des Control-Befehls durch einen Nutzer zu ermöglichen. Über einen HTTP-Post an den von Ihnen implementierten Restful Web Service, soll das Senden des Control-Befehls an den MQTT Broker initiiert werden. Die **Wago 750** SPS ist bereits so programmiert, dass sie konstant das genannte Topic überwacht und auf eingehende Nachrichten reagiert. Passen Sie daher ihren Restful Web Service so an, dass eine Kommunikation mit dem MQTT Broker möglich ist, um diesen Anwendungsfall zu implementieren.

Ein Beispiel für das grafische User Interfaces der **Wago 750** SPS mit blauen Buttons finden Sie auf der letzten Seite der Aufgabenstellung.

Beispiel für die grafischen User Interfaces

SPS: Wago 750



SPS: S7-1500

Ist-Wert: 27.309999465942383
Soll-Wert: 27.260000228881836
Differenz-Wert: 0.049999237060546875